q Kalman Filter Theory and Applications Heuristic Overview

Michael L. Carroll

July 1, 2020

©2018 by Michael L. Carroll

Part I
The Basic Kalman Equations
Topics

- Understanding the Equations: Heuristic Introduction

- Understanding the Equations: Heuristic Introduction
- Equation Drilldown: Taking the Equations Apart

- Understanding the Equations: Heuristic Introduction
- Equation Drilldown: Taking the Equations Apart
- State Space Concepts

- Understanding the Equations: Heuristic Introduction
- Equation Drilldown: Taking the Equations Apart
- State Space Concepts

- Recursive Predictor-Corrector Algorithms

- Recursive Predictor-Corrector Algorithms
- Running Averages

- Recursive Predictor-Corrector Algorithms
- Running Averages
- Extrapolation (Prediction)

- Recursive Predictor-Corrector Algorithms
- Running Averages
- Extrapolation (Prediction)
- Gain Computation

- Recursive Predictor-Corrector Algorithms
- Running Averages
- Extrapolation (Prediction)
- Gain Computation
- Measurement Update (Correction)

- Recursive Predictor-Corrector Algorithms
- Running Averages
- Extrapolation (Prediction)
- Gain Computation
- Measurement Update (Correction)

Recursive Predictor-Corrector Algorithm

What kind of problems are we trying to solve?

Recursive Predictor-Corrector Algorithm

What kind of problems are we trying to solve?

- To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements

Recursive Predictor-Corrector Algorithm

What kind of problems are we trying to solve?

- To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements
  - Generally a vector quantity: the **state** of the system or process

- To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements
  - Generally a vector quantity: the **state** of the system or process
  - Dynamic process is only modeled; not known perfectly and may be stochastic: i.e., slightly different each time it happens

- To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements
  - Generally a vector quantity: the **state** of the system or process
  - Dynamic process is only modeled; not known perfectly and may be stochastic: i.e., slightly different each time it happens
  - We never really know the true state at any given time – that's why we need an estimator!

Recursive Predictor-Corrector Algorithm

What kind of problems are we trying to solve?

- To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements
  - Generally a vector quantity: the **state** of the system or process
  - Dynamic process is only modeled; not known perfectly and may be stochastic: i.e., slightly different each time it happens
  - We never really know the true state at any given time – that's why we need an estimator!
- Measurements may be from multiple sensors simultaneously and/or sequentially

Recursive Predictor-Corrector Algorithm

What kind of problems are we trying to solve?

- To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements
  - Generally a vector quantity: the **state** of the system or process
  - Dynamic process is only modeled; not known perfectly and may be stochastic: i.e., slightly different each time it happens
  - We never really know the true state at any given time – that's why we need an estimator!
- Measurements may be from multiple sensors simultaneously and/or sequentially
  - Measurements don't necessarily measure the states directly

Recursive Predictor-Corrector Algorithm

What kind of problems are we trying to solve?

- To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements
  - Generally a vector quantity: the **state** of the system or process
  - Dynamic process is only modeled; not known perfectly and may be stochastic: i.e., slightly different each time it happens
  - We never really know the true state at any given time – that's why we need an estimator!
- Measurements may be from multiple sensors simultaneously and/or sequentially
  - Measurements don't necessarily measure the states directly

---

Recursive Predictor-Corrector Algorithm

Recursive Predictor-Corrector Algorithm

- Goal: Devise a method to combine several measurements and produce the best estimate of quantities of interest (the state of some system)

Recursive Predictor-Corrector Algorithm

- Goal: Devise a method to combine several measurements and produce the best estimate of quantities of interest (the state of some system)
- Measurements don't always happen simultaneously. Often sequential, especially if multiple, independent measurement sources are available.

Recursive Predictor-Corrector Algorithm

- Goal: Devise a method to combine several measurements and produce the best estimate of quantities of interest (the state of some system)
- Measurements don't always happen simultaneously. Often sequential, especially if multiple, independent measurement sources are available.
    - Hence we need an algorithm.

Recursive Predictor-Corrector Algorithm

- Goal: Devise a method to combine several measurements and produce the best estimate of quantities of interest (the state of some system)
- Measurements don't always happen simultaneously. Often sequential, especially if multiple, independent measurement sources are available.
  - Hence we need an algorithm.
  - Preferably a recursive algorithm whereby we update (correct) our last estimate with each new piece of measurement data

Recursive Predictor-Corrector Algorithm

- Goal: Devise a method to combine several measurements and produce the best estimate of quantities of interest (the state of some system)
- Measurements don't always happen simultaneously. Often sequential, especially if multiple, independent measurement sources are available.
  - Hence we need an algorithm.
  - Preferably a recursive algorithm whereby we update (correct) our last estimate with each new piece of measurement data
- But the system whose state we want to estimate is often dynamic, i.e., changing with time. So we need to take that into account (prediction).

Recursive Predictor-Corrector Algorithm

- Goal: Devise a method to combine several measurements and produce the best estimate of quantities of interest (the state of some system)
- Measurements don't always happen simultaneously. Often sequential, especially if multiple, independent measurement sources are available.
  - Hence we need an algorithm.
  - Preferably a recursive algorithm whereby we update (correct) our last estimate with each new piece of measurement data
- But the system whose state we want to estimate is often dynamic, i.e., changing with time. So we need to take that into account (prediction).
- Bottom line: We seek a recursive predictor-corrector algorithm.

Recursive Predictor-Corrector Algorithm

- Goal: Devise a method to combine several measurements and produce the best estimate of quantities of interest (the state of some system)
- Measurements don't always happen simultaneously. Often sequential, especially if multiple, independent measurement sources are available.
  - Hence we need an algorithm.
  - Preferably a recursive algorithm whereby we update (correct) our last estimate with each new piece of measurement data
- But the system whose state we want to estimate is often dynamic, i.e., changing with time. So we need to take that into account (prediction).
- Bottom line: We seek a recursive predictor-corrector algorithm.

Recursive Predictor-Corrector Algorithm

Recursive Predictor-Corrector Algorithm

- Prediction:

Recursive Predictor-Corrector Algorithm

- Prediction:
  - **Extrapolate** solution from previous iteration to present time

Recursive Predictor-Corrector Algorithm

- Prediction:
  - **Extrapolate** solution from previous iteration to present time (This is how we take system dynamics into account!)

Recursive Predictor-Corrector Algorithm

- Prediction:
  - **Extrapolate** solution from previous iteration to present time (This is how we take system dynamics into account!)
- Correction:

Recursive Predictor-Corrector Algorithm

- Prediction:
  - **Extrapolate** solution from previous iteration to present time (This is how we take system dynamics into account!)
- Correction:
  - Use measurement information to **update** or **correct** the prediction

Recursive Predictor-Corrector Algorithm

- Prediction:
  - **Extrapolate** solution from previous iteration to present time (This is how we take system dynamics into account!)
- Correction:
  - Use measurement information to **update** or **correct** the prediction
- And the cycle recursively begins anew!

Recursive Predictor-Corrector Algorithm

- Prediction:
  - **Extrapolate** solution from previous iteration to present time (This is how we take system dynamics into account!)
- Correction:
  - Use measurement information to **update** or **correct** the prediction
- And the cycle recursively begins anew!

- Time-dependent scalar or vector quantities: $x(k)$ (discrete), $x(t)$ (continuous)

- Time-dependent scalar or vector quantities: $x(k)$ (discrete), $x(t)$ (continuous)
- Corresponding vector components: $x_i(k)$ or $x_i(t)$

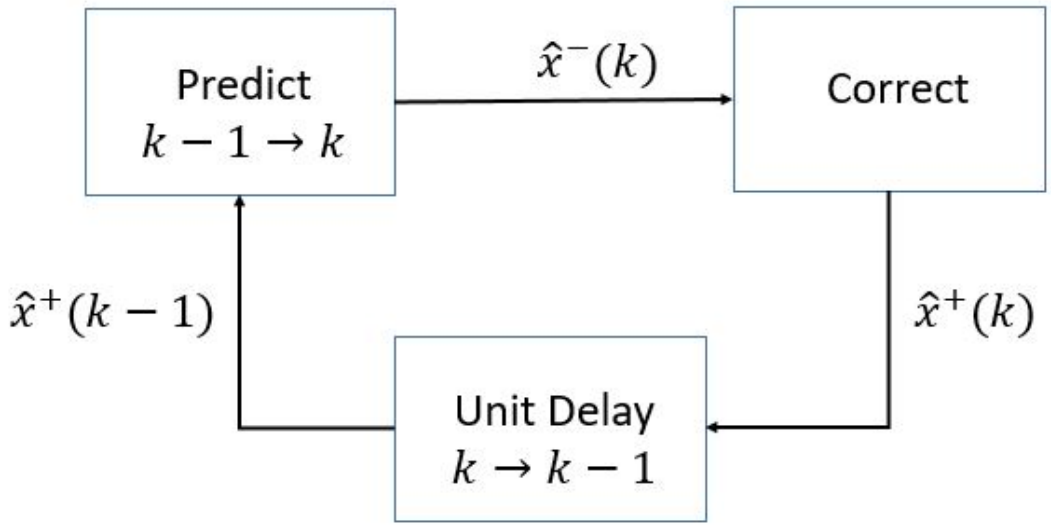
- Time-dependent scalar or vector quantities:
  $x(k)$ (discrete), $x(t)$ (continuous)
- Corresponding vector components:
  $x_i(k)$ or $x_i(t)$
- Time-dependent matrix quantities (capital letters):
  $P(k)$ (discrete), $P(t)$ (continuous)

- Time-dependent scalar or vector quantities:
  $x(k)$ (discrete), $x(t)$ (continuous)
- Corresponding vector components:
  $x_i(k)$ or $x_i(t)$
- Time-dependent matrix quantities (capital letters):
  $P(k)$ (discrete), $P(t)$ (continuous)
- Corresponding matrix components (elements):
  $p_{ij}(k)$ or $p_{ij}(t)$

- Time-dependent scalar or vector quantities:
  $x(k)$ (discrete), $x(t)$ (continuous)
- Corresponding vector components:
  $x_i(k)$ or $x_i(t)$
- Time-dependent matrix quantities (capital letters):
  $P(k)$ (discrete), $P(t)$ (continuous)
- Corresponding matrix components (elements):
  $p_{ij}(k)$ or $p_{ij}(t)$
- Use hats for estimated quantities and special superscripts to indicate pre- and post-update values:
  $\hat{x}^+(k)$ and $P^-(k)$

- Time-dependent scalar or vector quantities:
  $x(k)$ (discrete), $x(t)$ (continuous)
- Corresponding vector components:
  $x_i(k)$ or $x_i(t)$
- Time-dependent matrix quantities (capital letters):
  $P(k)$ (discrete), $P(t)$ (continuous)
- Corresponding matrix components (elements):
  $p_{ij}(k)$ or $p_{ij}(t)$
- Use hats for estimated quantities and special superscripts to indicate pre- and post-update values:
  $\hat{x}^+(k)$ and $P^-(k)$

Recursive Predictor-Corrector Algorithm (Discrete System)



Recursive Predictor-Corrector Algorithm

Recursive Predictor-Corrector Algorithm

- Algorithm

Recursive Predictor-Corrector Algorithm

- Algorithm
  - Computational procedure

Recursive Predictor-Corrector Algorithm

- Algorithm
  - Computational procedure
  - Given initial values of some quantities, new quantities are computed

Recursive Predictor-Corrector Algorithm

- Algorithm
  - Computational procedure
  - Given initial values of some quantities, new quantities are computed
- Recursive

Recursive Predictor-Corrector Algorithm

- Algorithm
  - Computational procedure
  - Given initial values of some quantities, new quantities are computed
- Recursive
  - The new quantities are the same as the old quantities

Recursive Predictor-Corrector Algorithm

- Algorithm
  - Computational procedure
  - Given initial values of some quantities, new quantities are computed
- Recursive
  - The new quantities are the same as the old quantities
  - New quantities become initial values for next cycle

Recursive Predictor-Corrector Algorithm

- Algorithm
  - Computational procedure
  - Given initial values of some quantities, new quantities are computed
- Recursive
  - The new quantities are the same as the old quantities
  - New quantities become initial values for next cycle

Simple Example: Running Averages

Simple Example: Running Averages

- Suppose we want to estimate some quantity like the resistance of a single resistor. We have ...

Simple Example: Running Averages

- Suppose we want to estimate some quantity like the resistance of a single resistor. We have ...
  - a voltmeter with which to make measurements

Simple Example: Running Averages

- Suppose we want to estimate some quantity like the resistance of a single resistor. We have ...
  - a voltmeter with which to make measurements
  - a model for the resistance: the "true" value x of the resistance is assumed to be constant in time: $x(k+1) = x(k)$

Simple Example: Running Averages

- Suppose we want to estimate some quantity like the resistance of a single resistor. We have ...
  - a voltmeter with which to make measurements
  - a model for the resistance: the "true" value x of the resistance is assumed to be constant in time: $x(k+1) = x(k)$
- We wish to take repeated measurements z of the resistance using the voltmeter

Simple Example: Running Averages

- Suppose we want to estimate some quantity like the resistance of a single resistor. We have ...
  - a voltmeter with which to make measurements
  - a model for the resistance: the "true" value x of the resistance is assumed to be constant in time: $x(k+1) = x(k)$
- We wish to take repeated measurements z of the resistance using the voltmeter $z(1), z(2), \ldots$

Simple Example: Running Averages

- Suppose we want to estimate some quantity like the resistance of a single resistor. We have ...
  - a voltmeter with which to make measurements
  - a model for the resistance: the "true" value x of the resistance is assumed to be constant in time: $x(k+1) = x(k)$
- We wish to take repeated measurements z of the resistance using the voltmeter $z(1), z(2), \ldots$

Simple Example: Running Averages

Simple Example: Running Averages

- The **state** that we are trying to estimate is the resistance value of the resistor

Simple Example: Running Averages

- The **state** that we are trying to estimate is the resistance value of the resistor
- We don't know what the state is. (Otherwise, we wouldn't need to take measurements)

Simple Example: Running Averages

- The **state** that we are trying to estimate is the resistance value of the resistor
- We don't know what the state is. (Otherwise, we wouldn't need to take measurements)
- Our **state dynamics** or **process model** is very simple: The state is 1-dimensional and does not change with time. So the model is:

$$x(k+1) = x(k)$$

Simple Example: Running Averages

- The **state** that we are trying to estimate is the resistance value of the resistor
- We don't know what the state is. (Otherwise, we wouldn't need to take measurements)
- Our **state dynamics** or **process model** is very simple: The state is 1-dimensional and does not change with time. So the model is:

$$x(k+1) = x(k)$$

Simple Example: Running Averages

Simple Example: Running Averages

- What is the best way to estimate the state of our system?

Simple Example: Running Averages

- What is the best way to estimate the state of our system?
- In the absence of any other information, we should probably just average the measurements

Simple Example: Running Averages

- What is the best way to estimate the state of our system?
- In the absence of any other information, we should probably just average the measurements
- We assume that the averages are unbiased, i.e., the measurement errors have zero mean

Simple Example: Running Averages

- What is the best way to estimate the state of our system?
- In the absence of any other information, we should probably just average the measurements
- We assume that the averages are unbiased, i.e., the measurement errors have zero mean

---

Simple Example: Running Averages

Simple Example: Running Averages

- Simple running average is like a Kalman filter with trivial dynamics ($x(k+1) = x(k)$)

Simple Example: Running Averages

- Simple running average is like a Kalman filter with trivial dynamics ($x(k+1) = x(k)$)

Given a sequence of random measurements,$z(1), z(2), \ldots$, compute at each stage the best estimate of the true value of the state: At the $n+1^{\text{st}}$ step, your best guess would be the average of all previous values:

Simple Example: Running Averages

- Simple running average is like a Kalman filter with trivial dynamics ($x(k+1) = x(k)$)

Given a sequence of random measurements,$z(1), z(2), \ldots$, compute at each stage the best estimate of the true value of the state: At the $n+1^{\text{st}}$ step, your best guess would be the average of all previous values:

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) = \frac{1}{k}[\, z(1) + z(2) + \cdots + z(k)\,] \qquad (1)$$

Simple Example: Running Averages

- Simple running average is like a Kalman filter with trivial dynamics ($x(k+1) = x(k)$)

Given a sequence of random measurements,$z(1), z(2), \ldots$, compute at each stage the best estimate of the true value of the state: At the $n+1^{\text{st}}$ step, your best guess would be the average of all previous values:

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) = \frac{1}{k}[\, z(1) + z(2) + \cdots + z(k)\,] \qquad (1)$$

Simple Example: Running Averages

Simple Example: Running Averages

- Q: Why is the average of all previous measurements the "best" estimate?

Simple Example: Running Averages

- Q: Why is the average of all previous measurements the "best" estimate?
- A: The dynamics tell us that the quantity to be estimated is not changing, and the only variability stems from the measurements, i.e., the measurement noise

Simple Example: Running Averages

- Q: Why is the average of all previous measurements the "best" estimate?
- A: The dynamics tell us that the quantity to be estimated is not changing, and the only variability stems from the measurements, i.e., the measurement noise
- Hence, we should try to eliminate the measurement variability by averaging the measurements

Simple Example: Running Averages

- Q: Why is the average of all previous measurements the "best" estimate?
- A: The dynamics tell us that the quantity to be estimated is not changing, and the only variability stems from the measurements, i.e., the measurement noise
- Hence, we should try to eliminate the measurement variability by averaging the measurements

---

Recursive Running Average

Recursive Running Average

- Therefore, as much as possible, we want to rid ourselves of the noise by averaging

Recursive Running Average

- Therefore, as much as possible, we want to rid ourselves of the noise by averaging
- Assume that noise is zero-mean, i.e., its average tends to zero over the long run

Recursive Running Average

- Therefore, as much as possible, we want to rid ourselves of the noise by averaging
- Assume that noise is zero-mean, i.e., its average tends to zero over the long run
- If we had a bias, we wouldn't call that noise; that would be a systematic error on the part of our voltmeter. We'd have to calibrate it out.

Recursive Running Average

- Therefore, as much as possible, we want to rid ourselves of the noise by averaging
- Assume that noise is zero-mean, i.e., its average tends to zero over the long run
- If we had a bias, we wouldn't call that noise; that would be a systematic error on the part of our voltmeter. We'd have to calibrate it out.
- Noise can't be calibrated out; we have to average it out over time

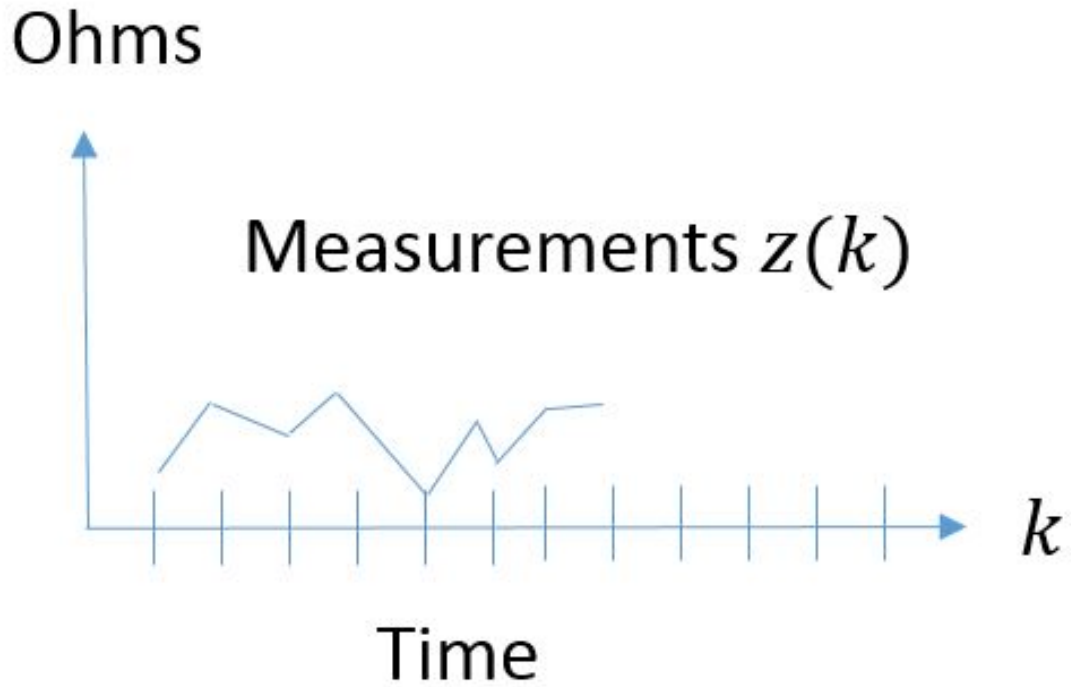
Recursive Running Average

- Therefore, as much as possible, we want to rid ourselves of the noise by averaging
- Assume that noise is zero-mean, i.e., its average tends to zero over the long run
- If we had a bias, we wouldn't call that noise; that would be a systematic error on the part of our voltmeter. We'd have to calibrate it out.
- Noise can't be calibrated out; we have to average it out over time

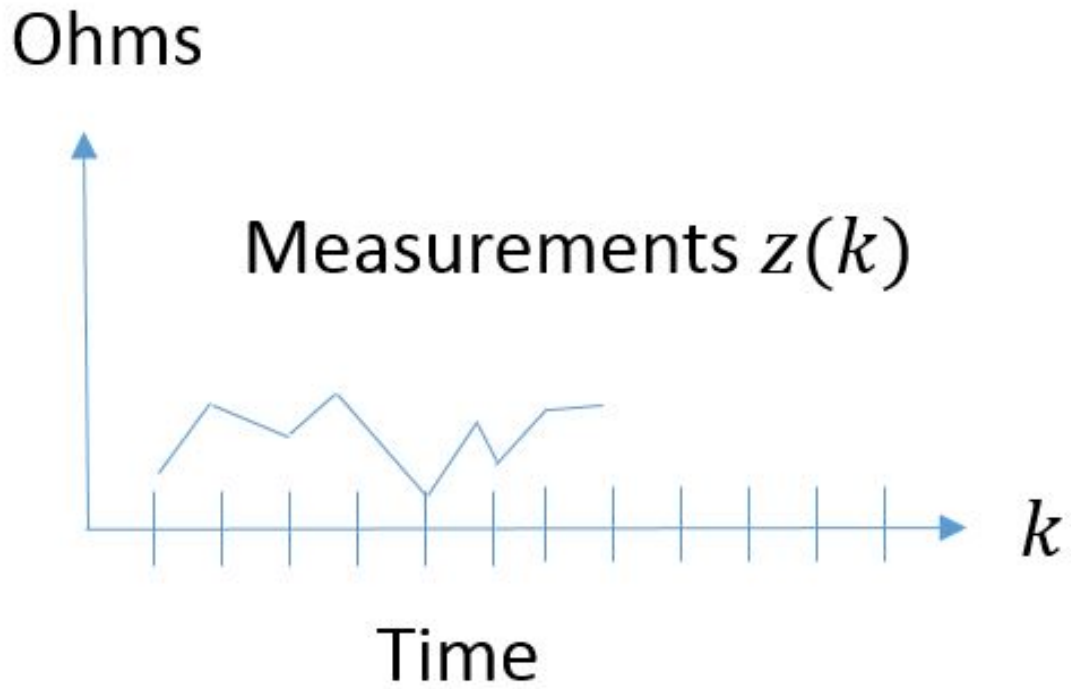
---

Recursive Running Average

Recursive Running Average



Recursive Running Average



Recursive Running Average

Recursive Running Average

(Eq.(1))

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) = \frac{1}{k}[\, z(1) + z(2) + \cdots + z(k)\,]$$

Recursive Running Average

(Eq.(1))

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) = \frac{1}{k}[\, z(1) + z(2) + \cdots + z(k)\,]$$

- Now suppose our estimating task is going to continue for a long time and we don't want to fill up our computer memory with a record of all the measurements taken since the start

Recursive Running Average

(Eq.(1))

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) = \frac{1}{k}[\, z(1) + z(2) + \cdots + z(k)\,]$$

- Now suppose our estimating task is going to continue for a long time and we don't want to fill up our computer memory with a record of all the measurements taken since the start
- We just want to use the previously calculated value and add a small adjustment

Recursive Running Average

(Eq.(1))

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) = \frac{1}{k}[\, z(1) + z(2) + \cdots + z(k)\,]$$

- Now suppose our estimating task is going to continue for a long time and we don't want to fill up our computer memory with a record of all the measurements taken since the start
- We just want to use the previously calculated value and add a small adjustment

---

Recursive Running Average

Derivation

- What's the difference between $\hat{x}(k+1)$ and $\hat{x}(k)$?

- What's the difference between $\hat{x}(k+1)$ and $\hat{x}(k)$?
- First we note that: $\sum_{j=1}^{k-1} z(j) = (k-1)\hat{x}(k)$

Recursive Running Average

Derivation

- What's the difference between $\hat{x}(k+1)$ and $\hat{x}(k)$?
- First we note that: $\sum_{j=1}^{k-1} z(j) = (k-1)\hat{x}(k)$
- Then $k\,\hat{x}(k+1) - (k-1)\hat{x}(k) = z(k)$

Recursive Running Average

Derivation

- What's the difference between $\hat{x}(k+1)$ and $\hat{x}(k)$?
- First we note that: $\sum_{j=1}^{k-1} z(j) = (k-1)\hat{x}(k)$
- Then $k\,\hat{x}(k+1) - (k-1)\hat{x}(k) = z(k)$
- Now we write use the definition for $\hat{x}(k+1)$ and replace the sum of $k-1$ terms as just indicated

- What's the difference between $\hat{x}(k+1)$ and $\hat{x}(k)$?
- First we note that: $\sum_{j=1}^{k-1} z(j) = (k-1)\hat{x}(k)$
- Then $k\,\hat{x}(k+1) - (k-1)\hat{x}(k) = z(k)$
- Now we write use the definition for $\hat{x}(k+1)$ and replace the sum of $k-1$ terms as just indicated

---

Another Step-by-Step Derivation:

Another Step-by-Step Derivation:

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) \qquad \text{by def. Eq. (1)}$$

Another Step-by-Step Derivation:

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) \qquad \text{by def. Eq. (1)}$$

$$= \frac{1}{k}\sum_{j=1}^{k-1} z(j) + \frac{1}{k} z(k) \qquad \text{separate last term from sum}$$

Another Step-by-Step Derivation:

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) \qquad \text{by def. Eq. (1)}$$

$$= \frac{1}{k}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{separate last term from sum}$$

$$= \frac{k-1}{k}\hat{x}(k) + \frac{1}{k}z(k) \qquad \text{replace the sum by def. Eq. (1) again}$$

Another Step-by-Step Derivation:

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) \qquad \text{by def. Eq. (1)}$$

$$= \frac{1}{k}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{separate last term from sum}$$

$$= \frac{k-1}{k}\hat{x}(k) + \frac{1}{k}z(k) \qquad \text{replace the sum by def. Eq. (1) again}$$

$$= \frac{k}{k}\hat{x}(k) - \frac{1}{k}\hat{x}(k) + \frac{1}{k}z(k) \qquad \text{distribute}$$

Another Step-by-Step Derivation:

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) \qquad \text{by def. Eq. (1)}$$

$$= \frac{1}{k}\sum_{j=1}^{k-1} z(j) + \frac{1}{k} z(k) \qquad \text{separate last term from sum}$$

$$= \frac{k-1}{k}\hat{x}(k) + \frac{1}{k} z(k) \qquad \text{replace the sum by def. Eq. (1) again}$$

$$= \frac{k}{k}\hat{x}(k) - \frac{1}{k}\hat{x}(k) + \frac{1}{k} z(k) \qquad \text{distribute}$$

$$= \hat{x}(k) + \frac{1}{k}\left[\, z(k) - \hat{x}(k) \,\right]. \qquad \text{simplify, rearrange and factor out } 1/k$$

Another Step-by-Step Derivation:

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) \qquad \text{by def. Eq. (1)}$$

$$= \frac{1}{k}\sum_{j=1}^{k-1} z(j) + \frac{1}{k} z(k) \qquad \text{separate last term from sum}$$

$$= \frac{k-1}{k}\hat{x}(k) + \frac{1}{k} z(k) \qquad \text{replace the sum by def. Eq. (1) again}$$

$$= \frac{k}{k}\hat{x}(k) - \frac{1}{k}\hat{x}(k) + \frac{1}{k} z(k) \qquad \text{distribute}$$

$$= \hat{x}(k) + \frac{1}{k}\left[\, z(k) - \hat{x}(k) \,\right]. \qquad \text{simplify, rearrange and factor out } 1/k$$

Another Step-by-Step Derivation:

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) \qquad \text{by def. Eq. (1)}$$

$$= \frac{1}{k}\sum_{j=1}^{k-1} z(j) + \frac{1}{k} z(k) \qquad \text{separate last term from sum}$$

$$= \frac{k-1}{k}\hat{x}(k) + \frac{1}{k} z(k) \qquad \text{replace the sum by def. Eq. (1) again}$$

$$= \frac{k}{k}\hat{x}(k) - \frac{1}{k}\hat{x}(k) + \frac{1}{k} z(k) \qquad \text{distribute}$$

$$= \hat{x}(k) + \frac{1}{k}\left[\, z(k) - \hat{x}(k) \,\right]. \qquad \text{simplify, rearrange and factor out } 1/k$$

Another Step-by-Step Derivation:

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) \qquad \text{by def. Eq. (1)}$$

$$= \frac{1}{k}\sum_{j=1}^{k-1} z(j) + \frac{1}{k} z(k) \qquad \text{separate last term from sum}$$

$$= \frac{k-1}{k}\hat{x}(k) + \frac{1}{k} z(k) \qquad \text{replace the sum by def. Eq. (1) again}$$

$$= \frac{k}{k}\hat{x}(k) - \frac{1}{k}\hat{x}(k) + \frac{1}{k} z(k) \qquad \text{distribute}$$

$$= \hat{x}(k) + \frac{1}{k}\left[\, z(k) - \hat{x}(k) \,\right]. \qquad \text{simplify, rearrange and factor out } 1/k$$

Recursive Running Average

Recursive Running Average

- We now have a recursive algorithm for computing a running average.
  Recipe:

Recursive Running Average

- We now have a recursive algorithm for computing a running average.
  Recipe:
  - Take new measurement, subtract off old average and weight the resulting residual difference by a *gain* of $1/k$

Recursive Running Average

- We now have a recursive algorithm for computing a running average.
  Recipe:
  - Take new measurement, subtract off old average and weight the resulting residual difference by a *gain* of $1/k$
  - Add this weighted residual to the old average and, voilà, you have the new average!

Recursive Running Average

- We now have a recursive algorithm for computing a running average.
  Recipe:
  - Take new measurement, subtract off old average and weight the resulting residual difference by a *gain* of $1/k$
  - Add this weighted residual to the old average and, voilà, you have the new average!

Recursive Running Average

Recursive Running Average

- Note that we don't have to remember all the previous values or averages, only the last average; we use the latest measurement to **correct** the last average

Recursive Running Average

- Note that we don't have to remember all the previous values or averages, only the last average; we use the latest measurement to **correct** the last average
- Note also that as k goes to ∞, the gain $1/k$ goes to 0. (The filter is said to be going to sleep!)

Recursive Running Average

- Note that we don't have to remember all the previous values or averages, only the last average; we use the latest measurement to **correct** the last average
- Note also that as k goes to ∞, the gain $1/k$ goes to 0. (The filter is said to be going to sleep!)

$$\hat{x}(k+1) = \hat{x}(k) + \frac{1}{k}\left[z(k) - \hat{x}(k)\right] \tag{2}$$

Recursive Running Average

- Note that we don't have to remember all the previous values or averages, only the last average; we use the latest measurement to **correct** the last average
- Note also that as k goes to ∞, the gain $1/k$ goes to 0. (The filter is said to be going to sleep!)

$$\hat{x}(k+1) = \hat{x}(k) + \frac{1}{k}\left[z(k) - \hat{x}(k)\right] \tag{2}$$

Recursive Running Average

Recursive Running Average

- The **residual** $z(k) - \hat{x}(k)$ is the difference between the actual new measurement and what we expected or predicted the new measurement to be

Recursive Running Average

- The **residual** $z(k) - \hat{x}(k)$ is the difference between the actual new measurement and what we expected or predicted the new measurement to be
- Why is $\hat{x}(k)$ our predicted measurement?

Recursive Running Average

- The **residual** $z(k) - \hat{x}(k)$ is the difference between the actual new measurement and what we expected or predicted the new measurement to be
- Why is $\hat{x}(k)$ our predicted measurement?
- Answer: In the absence of new information, the previous estimate is the only thing we have to go on!

Recursive Running Average

- The **residual** $z(k) - \hat{x}(k)$ is the difference between the actual new measurement and what we expected or predicted the new measurement to be
- Why is $\hat{x}(k)$ our predicted measurement?
- Answer: In the absence of new information, the previous estimate is the only thing we have to go on!
  - When we take state dynamics into account, this will make the predicted measurement different than our previous estimate

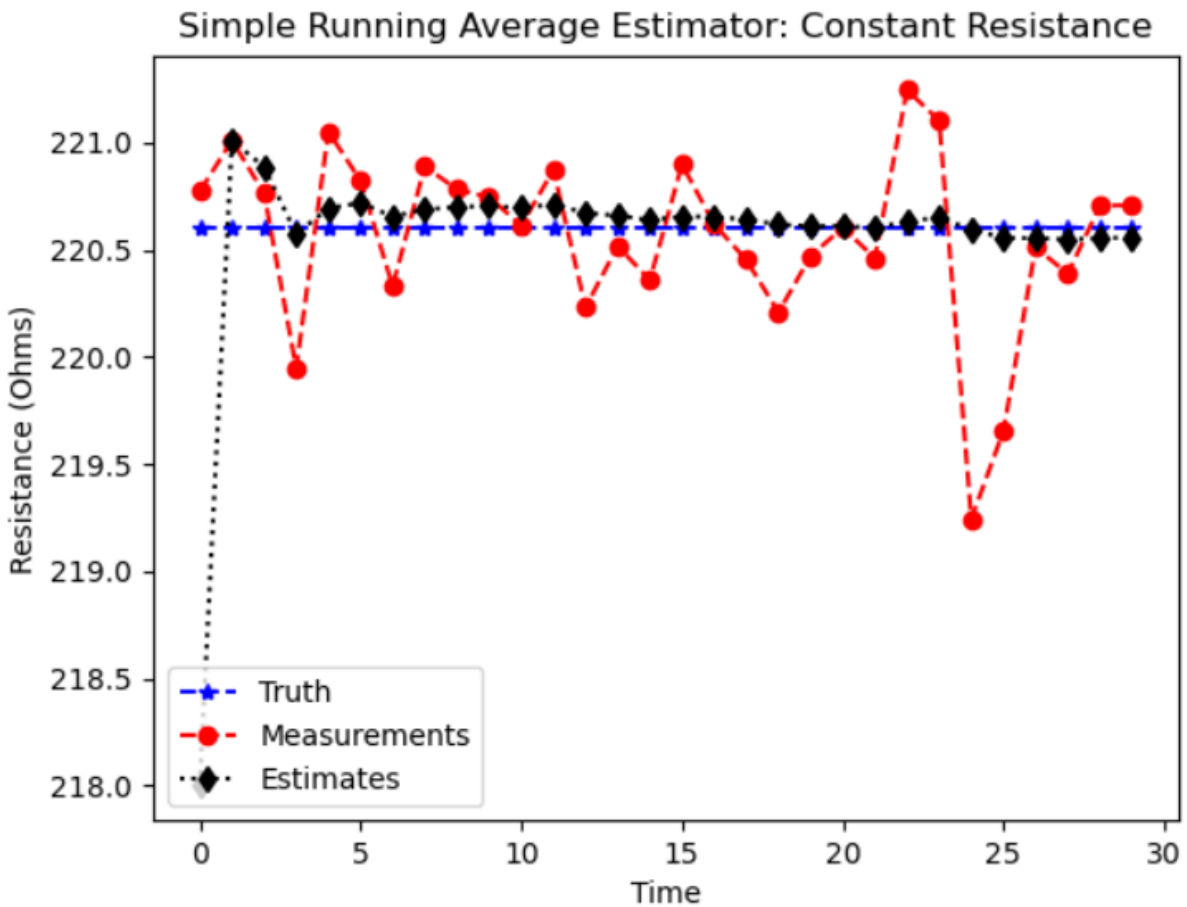
Recursive Running Average

- The **residual** $z(k) - \hat{x}(k)$ is the difference between the actual new measurement and what we expected or predicted the new measurement to be
- Why is $\hat{x}(k)$ our predicted measurement?
- Answer: In the absence of new information, the previous estimate is the only thing we have to go on!
  - When we take state dynamics into account, this will make the predicted measurement different than our previous estimate

---

Recursive Running Average Python Scripts: run_avg_demo.py

Recursive Running Average Python Scripts: run_avg_demo.py



- Uses (implicitly) all data points, even those from the "remote" past

- Uses (implicitly) all data points, even those from the "remote" past
- Weights each data point the same

- Uses (implicitly) all data points, even those from the "remote" past
- Weights each data point the same
- Gain gets smaller and smaller ... running average filter goes to sleep, becomes insensitive to new information

- Uses (implicitly) all data points, even those from the "remote" past
- Weights each data point the same
- Gain gets smaller and smaller ... running average filter goes to sleep, becomes insensitive to new information
- Can improve things with a simple moving average (sliding window)

- Uses (implicitly) all data points, even those from the "remote" past
- Weights each data point the same
- Gain gets smaller and smaller ... running average filter goes to sleep, becomes insensitive to new information
- Can improve things with a simple moving average (sliding window)

- Simple Moving Average of Window Size M only averages the M most recent measurements

- Simple Moving Average of Window Size M only averages the M most recent measurements
- Has a fixed gain of $1/M$ which never goes to zero

- Simple Moving Average of Window Size M only averages the M most recent measurements
- Has a fixed gain of $1/M$ which never goes to zero
- Assume the number of available data points is $N > M$

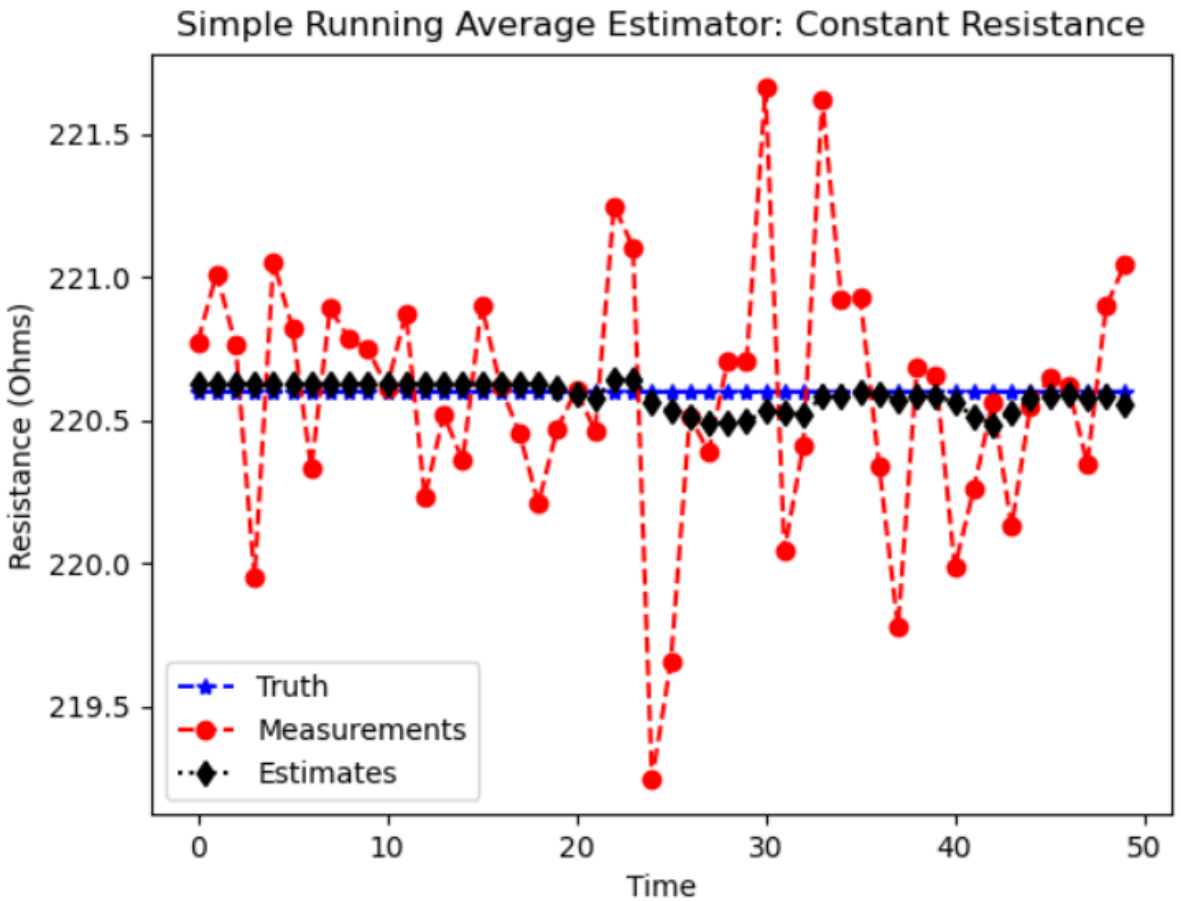
- Simple Moving Average of Window Size M only averages the M most recent measurements
- Has a fixed gain of $1/M$ which never goes to zero
- Assume the number of available data points is $N > M$
- Simple Moving Average of window size M is $s(k, M) = \frac{1}{M} \sum_{j=k-M+1}^{k} z(j)$, valid for $k \geq M - 1$ if indexing starts at 0.

- Simple Moving Average of Window Size M only averages the M most recent measurements
- Has a fixed gain of $1/M$ which never goes to zero
- Assume the number of available data points is $N > M$
- Simple Moving Average of window size M is $s(k, M) = \frac{1}{M} \sum_{j=k-M+1}^{k} z(j)$, valid for $k \geq M - 1$ if indexing starts at 0.
- Recursive SMA formula: $s(k+1, M) := s(k, M) + \frac{1}{M} [z(k) - z(k - M + 1)]$

- Simple Moving Average of Window Size M only averages the M most recent measurements
- Has a fixed gain of $1/M$ which never goes to zero
- Assume the number of available data points is $N > M$
- Simple Moving Average of window size M is $s(k, M) = \frac{1}{M} \sum_{j=k-M+1}^{k} z(j)$, valid for $k \geq M - 1$ if indexing starts at 0.
- Recursive SMA formula: $s(k+1, M) := s(k, M) + \frac{1}{M} [z(k) - z(k - M + 1)]$

Simple Moving Average Python Scripts: simple_moving_avg_demo.py

Simple Moving Average Python Scripts: simple_moving_avg_demo.py



- Note that the recursive SMA seems to have shortcomings of its own

- Note that the recursive SMA seems to have shortcomings of its own
- First, it seems overly conservative about updating the previous SMA value

- Note that the recursive SMA seems to have shortcomings of its own
- First, it seems overly conservative about updating the previous SMA value
  - However, this is because unchanging process dynamics means that measurements do not change much

- Note that the recursive SMA seems to have shortcomings of its own
- First, it seems overly conservative about updating the previous SMA value
  - However, this is because unchanging process dynamics means that measurements do not change much
  - Performs worse than running average on constant dynamics, but better with changing dynamics (See next slide)

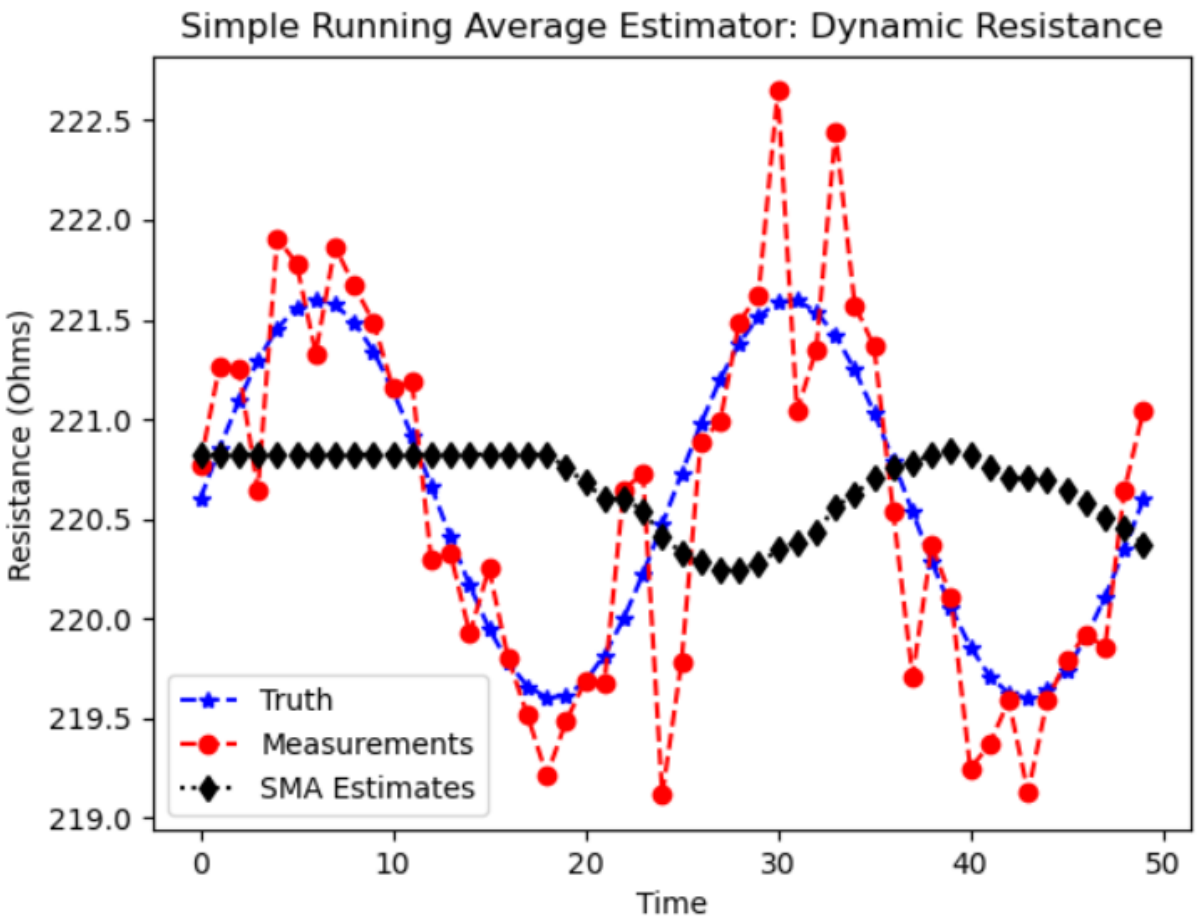
- Note that the recursive SMA seems to have shortcomings of its own
- First, it seems overly conservative about updating the previous SMA value
  - However, this is because unchanging process dynamics means that measurements do not change much
  - Performs worse than running average on constant dynamics, but better with changing dynamics (See next slide)
- A more aggresive formulation of the recursive SMA can be defined by using a weighting scheme whereby the oldest measurement is weighted less than the most recent

- Note that the recursive SMA seems to have shortcomings of its own
- First, it seems overly conservative about updating the previous SMA value
  - However, this is because unchanging process dynamics means that measurements do not change much
  - Performs worse than running average on constant dynamics, but better with changing dynamics (See next slide)
- A more aggresive formulation of the recursive SMA can be defined by using a weighting scheme whereby the oldest measurement is weighted less than the most recent
- Neither of these averaging filters explicitly takes into account the degree of uncertainty in the process dynamics

- Note that the recursive SMA seems to have shortcomings of its own
- First, it seems overly conservative about updating the previous SMA value
  - However, this is because unchanging process dynamics means that measurements do not change much
  - Performs worse than running average on constant dynamics, but better with changing dynamics (See next slide)
- A more aggresive formulation of the recursive SMA can be defined by using a weighting scheme whereby the oldest measurement is weighted less than the most recent
- Neither of these averaging filters explicitly takes into account the degree of uncertainty in the process dynamics

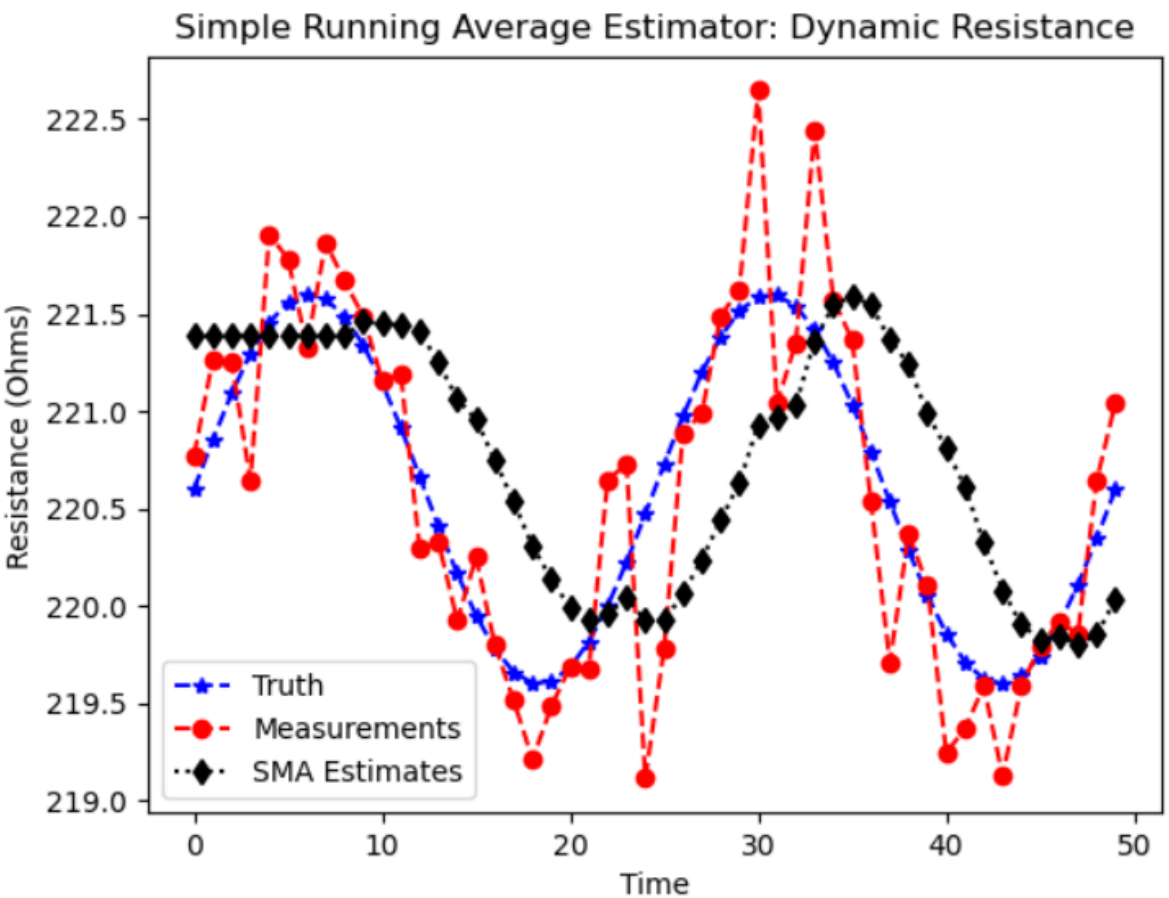
Simple Moving Average Python Scripts: dyn_simple_moving_avg_demo.py with window size of 20

Simple Moving Average Python Scripts: dyn_simple_moving_avg_demo.py with window size of 20



Simple Moving Average Python Scripts: dyn_simple_moving_avg_demo.py with window size of 10

Simple Moving Average Python Scripts: dyn_simple_moving_avg_demo.py with window size of 10



Extrapolation (Prediction)

Extrapolation (Prediction)

- In the running average example, the dynamics were trivial, i.e., $x(k+1) = x(k)$

Extrapolation (Prediction)

- In the running average example, the dynamics were trivial, i.e., $x(k+1) = x(k)$
- This means that the state never really changes. It's always equal to its previous value.

Extrapolation (Prediction)

- In the running average example, the dynamics were trivial, i.e., $x(k+1) = x(k)$
- This means that the state never really changes. It's always equal to its previous value.
- We just don't know what the exact value is.

Extrapolation (Prediction)

- In the running average example, the dynamics were trivial, i.e., $x(k+1) = x(k)$
- This means that the state never really changes. It's always equal to its previous value.
- We just don't know what the exact value is.
- Example: Measuring the same resistor over and over again with a voltmeter to estimate the resistor's resistance value. Or getting on and off a scale repeatedly.

Extrapolation (Prediction)

- In the running average example, the dynamics were trivial, i.e., $x(k+1) = x(k)$
- This means that the state never really changes. It's always equal to its previous value.
- We just don't know what the exact value is.
- Example: Measuring the same resistor over and over again with a voltmeter to estimate the resistor's resistance value. Or getting on and off a scale repeatedly.

Extrapolation (Prediction)

Extrapolation (Prediction)

- In more general Kalman filtering, we complicate the situation by adding two more elements:

Extrapolation (Prediction)

- In more general Kalman filtering, we complicate the situation by adding two more elements:
  - State dynamics with time dependence and process noise:

$$x(k+1) = f\left[k, x(k), w(k)\right]$$

Extrapolation (Prediction)

- In more general Kalman filtering, we complicate the situation by adding two more elements:
  - State dynamics with time dependence and process noise:
$$x(k+1) = f\left[k, x(k), w(k)\right]$$
  - Typically, the process noise is additive:
$$x(k+1) = f\left[k, x(k)\right] + w(k)$$

Extrapolation (Prediction)

- In more general Kalman filtering, we complicate the situation by adding two more elements:
  - State dynamics with time dependence and process noise:
$$x(k+1) = f\left[k, x(k), w(k)\right]$$
  - Typically, the process noise is additive:
$$x(k+1) = f\left[k, x(k)\right] + w(k)$$

Extrapolation (Prediction)

Extrapolation (Prediction)

- Form of function f can be simple or complicated, linear or nonlinear

Extrapolation (Prediction)

- Form of function f can be simple or complicated, linear or nonlinear
- Independent variable k indicates possible explicit time dependence

Extrapolation (Prediction)

- Form of function f can be simple or complicated, linear or nonlinear
- Independent variable k indicates possible explicit time dependence
- Process noise assumed to be Gaussian white noise (to be defined later)

Extrapolation (Prediction)

- Form of function f can be simple or complicated, linear or nonlinear
- Independent variable k indicates possible explicit time dependence
- Process noise assumed to be Gaussian white noise (to be defined later)

Extrapolation (Prediction)

Extrapolation (Prediction)

- General Kalman filtering further complicates things by allowing the state variable to be vector-valued

Extrapolation (Prediction)

- General Kalman filtering further complicates things by allowing the state variable to be vector-valued
- This turns the function f into a vector-valued function, often a matrix (possibly time-dependent)

Extrapolation (Prediction)

- General Kalman filtering further complicates things by allowing the state variable to be vector-valued
- This turns the function f into a vector-valued function, often a matrix (possibly time-dependent)
- Process noise must therefore also become a vector-valued variable

Extrapolation (Prediction)

- General Kalman filtering further complicates things by allowing the state variable to be vector-valued
- This turns the function f into a vector-valued function, often a matrix (possibly time-dependent)
- Process noise must therefore also become a vector-valued variable

---

Extrapolation (Prediction)

Extrapolation (Prediction)

- Note that presence of a random forcing function like Gaussian white noise makes the state vector process a stochastic process (aka random process)

Extrapolation (Prediction)

- Note that presence of a random forcing function like Gaussian white noise makes the state vector process a stochastic process (aka random process)
  - If white noise were zero, problem would be deterministic

Extrapolation (Prediction)

- Note that presence of a random forcing function like Gaussian white noise makes the state vector process a stochastic process (aka random process)
  - If white noise were zero, problem would be deterministic
- We'll see what this means when we take a look at random variables and random processes

Extrapolation (Prediction)

- Note that presence of a random forcing function like Gaussian white noise makes the state vector process a stochastic process (aka random process)
  - If white noise were zero, problem would be deterministic
- We'll see what this means when we take a look at random variables and random processes
- Even for the same initial conditions the sample trajectory will be different each time, due to randomly changing forcing functions

Extrapolation (Prediction)

- Note that presence of a random forcing function like Gaussian white noise makes the state vector process a stochastic process (aka random process)
  - If white noise were zero, problem would be deterministic
- We'll see what this means when we take a look at random variables and random processes
- Even for the same initial conditions the sample trajectory will be different each time, due to randomly changing forcing functions

---

Extrapolation (Prediction)

Extrapolation (Prediction)

- Note that we did not mention a control vector u that could also be part of the forcing function

Extrapolation (Prediction)

- Note that we did not mention a control vector u that could also be part of the forcing function
  - Control vectors usually considered to be deterministic although in reality you can't control things perfectly!

Extrapolation (Prediction)

- Note that we did not mention a control vector u that could also be part of the forcing function
  - Control vectors usually considered to be deterministic although in reality you can't control things perfectly!
  - E.g., aircraft actuator (e.g., elevator or flaps) deflection angle might be a control quantity yet its precise value may not be known!

Extrapolation (Prediction)

- Note that we did not mention a control vector u that could also be part of the forcing function
  - Control vectors usually considered to be deterministic although in reality you can't control things perfectly!
  - E.g., aircraft actuator (e.g., elevator or flaps) deflection angle might be a control quantity yet its precise value may not be known!
- We are going to ignore the presence of deterministic control and focus solely on state estimation

Extrapolation (Prediction)

- Note that we did not mention a control vector u that could also be part of the forcing function
  - Control vectors usually considered to be deterministic although in reality you can't control things perfectly!
  - E.g., aircraft actuator (e.g., elevator or flaps) deflection angle might be a control quantity yet its precise value may not be known!
- We are going to ignore the presence of deterministic control and focus solely on state estimation

---

Measurement Update (Correction)

Measurement Update (Correction)

- In running average example, the update implicitly assumed that measurements were corrupted by white noise

Measurement Update (Correction)

- In running average example, the update implicitly assumed that measurements were corrupted by white noise
- Math model for this would be:
  $z(k) = x(k) + v(k)$,
  where $v(k)$ is Gaussian white measurement noise sequence

Measurement Update (Correction)

- In running average example, the update implicitly assumed that measurements were corrupted by white noise
- Math model for this would be:
  $z(k) = x(k) + v(k)$,
  where $v(k)$ is Gaussian white measurement noise sequence
- Measurement processing is also often called Observation processing

Measurement Update (Correction)

- In running average example, the update implicitly assumed that measurements were corrupted by white noise
- Math model for this would be:
  $z(k) = x(k) + v(k)$,
  where $v(k)$ is Gaussian white measurement noise sequence
- Measurement processing is also often called Observation processing

---

Measurement Update (Correction)

Measurement Update (Correction)

- Because we assumed that measurement noise was white Gaussian and zero mean, our best guess was simply the average value of all the measurements

Measurement Update (Correction)

- Because we assumed that measurement noise was white Gaussian and zero mean, our best guess was simply the average value of all the measurements
- Remember the resistor example: our voltmeter is not perfect, and each measurement will differ slightly from the previous one, even though we are measuring the same resistor each time

Measurement Update (Correction)

- Because we assumed that measurement noise was white Gaussian and zero mean, our best guess was simply the average value of all the measurements
- Remember the resistor example: our voltmeter is not perfect, and each measurement will differ slightly from the previous one, even though we are measuring the same resistor each time

---

Measurement Update (Correction)

Measurement Update (Correction)

- General Kalman filtering complicates the measurement model by adding two new elements:

Measurement Update (Correction)

- General Kalman filtering complicates the measurement model by adding two new elements:
  - The measurement may have a more complicated relationship to the underlying system state:

$$z(k) = h\left[k, x(k)\right] + v(k),$$

where h could be a non-linear function with possible explicit time dependence

Measurement Update (Correction)

- General Kalman filtering complicates the measurement model by adding two new elements:
  - The measurement may have a more complicated relationship to the underlying system state:

    $$z(k) = h[k, x(k)] + v(k),$$

    where h could be a non-linear function with possible explicit time dependence
  - Just as the state x is allowed to be a vector variable, so the measurement z could also be a vector variable
    This means that you can measure several quantities simultaneously, or have redundant measurements

Measurement Update (Correction)

- General Kalman filtering complicates the measurement model by adding two new elements:
  - The measurement may have a more complicated relationship to the underlying system state:

    $$z(k) = h[k, x(k)] + v(k),$$

    where h could be a non-linear function with possible explicit time dependence
  - Just as the state x is allowed to be a vector variable, so the measurement z could also be a vector variable
    This means that you can measure several quantities simultaneously, or have redundant measurements

Kalman Gain Computation

Kalman Gain Computation

- Even though we've discussed the measurement update processing step, there is an important calculation to make just prior to making the measurement update

Kalman Gain Computation

- Even though we've discussed the measurement update processing step, there is an important calculation to make just prior to making the measurement update
- Before we can incorporate the information contained in both the old estimate and the new measurement, we have to first calculate the **Kalman gain** $K(k)$ at the kth step

Kalman Gain Computation

- Even though we've discussed the measurement update processing step, there is an important calculation to make just prior to making the measurement update
- Before we can incorporate the information contained in both the old estimate and the new measurement, we have to first calculate the **Kalman gain** $K(k)$ at the kth step
- $K(k) = \frac{1}{k}$ in our running average example

Kalman Gain Computation

- Even though we've discussed the measurement update processing step, there is an important calculation to make just prior to making the measurement update
- Before we can incorporate the information contained in both the old estimate and the new measurement, we have to first calculate the **Kalman gain** $K(k)$ at the kth step
- $K(k) = \frac{1}{k}$ in our running average example
- If we have k pieces of information *of equal quality*, it seems intuitive that we weight each piece by $1/k$

Kalman Gain Computation

- Even though we've discussed the measurement update processing step, there is an important calculation to make just prior to making the measurement update
- Before we can incorporate the information contained in both the old estimate and the new measurement, we have to first calculate the **Kalman gain** $K(k)$ at the kth step
- $K(k) = \frac{1}{k}$ in our running average example
- If we have k pieces of information *of equal quality*, it seems intuitive that we weight each piece by $1/k$

Kalman Gain Computation

Kalman Gain Computation

- What we are weighting is the measurement residual, also called the **innovation**: $z(k) - \hat{x}(k)$, the difference between the actual measurement and the predicted measurement.

Kalman Gain Computation

- What we are weighting is the measurement residual, also called the **innovation**: $z(k) - \hat{x}(k)$, the difference between the actual measurement and the predicted measurement.
- The innovation contains all the new information, i.e., the new measurement, minus what we thought we already knew, i.e., the previous estimate

Kalman Gain Computation

- What we are weighting is the measurement residual, also called the **innovation**: $z(k) - \hat{x}(k)$, the difference between the actual measurement and the predicted measurement.
- The innovation contains all the new information, i.e., the new measurement, minus what we thought we already knew, i.e., the previous estimate
- The Kalman gain computation occurs after the extrapolation but before we update the estimate with measurement information

Kalman Gain Computation

- What we are weighting is the measurement residual, also called the **innovation**: $z(k) - \hat{x}(k)$, the difference between the actual measurement and the predicted measurement.
- The innovation contains all the new information, i.e., the new measurement, minus what we thought we already knew, i.e., the previous estimate
- The Kalman gain computation occurs after the extrapolation but before we update the estimate with measurement information
- The Kalman gain is also sometimes referred to as the 'Innovation gain'

Kalman Gain Computation

- What we are weighting is the measurement residual, also called the **innovation**: $z(k) - \hat{x}(k)$, the difference between the actual measurement and the predicted measurement.
- The innovation contains all the new information, i.e., the new measurement, minus what we thought we already knew, i.e., the previous estimate
- The Kalman gain computation occurs after the extrapolation but before we update the estimate with measurement information
- The Kalman gain is also sometimes referred to as the 'Innovation gain'

Exercises

1. Let's re-familiarize ourselves with vectors and matrices. Addition:

$$\begin{bmatrix} x_2 \\ x_n \end{bmatrix} + \begin{bmatrix} y_2 \\ y_n \end{bmatrix} = \begin{bmatrix} x_1 + y_1 \\ x_2 + y_2 \\ x_n + y_n \end{bmatrix}$$

For vectors in three space, this would be:

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = x_3 + y_3$$

Compute:

$$\begin{bmatrix} -1 \\ 3 \\ 0 \end{bmatrix} + \begin{bmatrix} 7 \\ -1 \\ 5 \end{bmatrix} =$$

Exercises

$$\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix} =$$

$$\begin{bmatrix} 0.75 \\ -3 \\ 100 \end{bmatrix} + \begin{bmatrix} 0.75 \\ 0 \\ -1000 \end{bmatrix} =$$

Exercises

$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} =$$

$$\begin{bmatrix} 1.1 \\ -3.2 \\ 9.7 \end{bmatrix} + \begin{bmatrix} -0.8 \\ 3.2 \\ 10 \end{bmatrix} =$$

Exercises

2. Multiplying a row vector times a column vector:

$$\begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} = a_1 b_1 + a_2 b_2 + a_3 b_3$$

Now you try it:

$$\begin{bmatrix} 0 & 1 & 3 \end{bmatrix} \begin{bmatrix} 4 \\ 3 \\ 0 \end{bmatrix} =$$

$$\begin{bmatrix} -1 & 3 & -2 \end{bmatrix} \begin{bmatrix} 1 \\ -2 \\ 2 \end{bmatrix} =$$

Exercises

$$\begin{bmatrix} x & y & z \end{bmatrix} \begin{bmatrix} u \\ v \\ w \end{bmatrix} =$$

$$\begin{bmatrix} 2x & .5x & -7x \end{bmatrix} \begin{bmatrix} -10 \\ 4 \\ -1 \end{bmatrix} =$$

Exercises

3. Multiplying a matrix times a column vector:

$$\begin{bmatrix} 1 & 0 & 1 \\ 2 & 7 & 0 \\ 3 & 2 & 5 \end{bmatrix} \begin{bmatrix} 7 \\ -1 \\ 5 \end{bmatrix} =$$

Hint: Multiply the first row of the matrix times the column vector to get the first element of the new column vector. Then multiply by the second row of the matrix, and so on.

$$\begin{bmatrix} 2 & 1 & 0 \\ 0 & 1 & 1 \\ 2 & 2 & 2 \end{bmatrix} \begin{bmatrix} 3 \\ -3 \\ 1 \end{bmatrix} =$$

$$\begin{bmatrix} 3 & 1 & 1 \\ -1 & 1 & -1 \\ 0 & 2 & 2 \end{bmatrix} \begin{bmatrix} 3 \\ -3 \\ 1 \end{bmatrix} =$$

Exercises

4. What happens if you multiply a column on the left by a row vector on the right?
   item What is the transpose of the following matrix?

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

Exercises

5. Given the following scalar measurements of some constant scalar process, calculate the average value at each stage k by two different methods: 1) by averaging all past values and 2) by using the recursive algorithm in Eq. (2):
    1. $z_1 = 0.76$
    2. $z_2 = 0.67$
    3. $z_3 = 0.79$
    4. $z_4 = 0.81$
    5. $z_5 = 0.78$
    6. $z_6 = 0.79$
    7. $z_7 = 0.81$
    8. $z_8 = 0.82$
    9. $z_9 = 0.77$

Exercises

6. You are traveling in a car at 45 mph in a 35 mph speed zone. Your friendly neighborhood police officer is watching you with her radar gun. She is measuring your speed, of course, but needs to report as well your position as a function of time. Write down a dynamics and measurement model for this problem in which the state variable is your positional displacement from some arbitrary initial position. Assume that there is no process noise in your dynamics and that your velocity is not changing. Assume the policeman makes a measurement with her radar gun every 2 seconds.
   Hint: You have two variables of interest!
7. How is this problem different from the resistor problem?